E-COMMERCE ORDER MANAGEMENT SYSTEM

DBMS Capstone Project - SQL Implementation

Batch	11 (AIML)
Academic Year	2026-2027
Guide	Dr P Kishorekumar, Associate Professor, Dept. of AIML
University	Aditya University
Team	P. Chandrika (25B11AI911) G. Anvitha (25B11AI332) J. Tanishka (25B11AI420) S. Sudhadevi (25B11AIB23)
Database	MySQL (MySQL Workbench)
Tables	CUSTOMER, CATEGORY, PRODUCT, ORDER, ORDER_ITEM, PAYMENT, DELIVERY

Contents

Part 1: SQL code (create tables, insert data, queries)
Part 2: Screenshots of implementation and output

Part 1: SQL Code

1. Create Database and Tables (DDL)

```
-- ============================================
-- E-commerce Order Management System (MySQL)
-- File 1: Database and table creation (DDL)
-- ============================================

DROP DATABASE IF EXISTS ecommerce_order_management;
CREATE DATABASE ecommerce_order_management;
USE ecommerce_order_management;

CREATE TABLE CUSTOMER (
    CUSTOMER_ID INT PRIMARY KEY,
    CUSTOMER_NAME VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Address VARCHAR(200)
);

CREATE TABLE CATEGORY (
    CATEGORY_ID INT PRIMARY KEY,
    CATEGORY_NAME VARCHAR(100) NOT NULL,
    Description VARCHAR(200)
);

CREATE TABLE PRODUCT (
    PRODUCT_ID INT PRIMARY KEY,
    PRODUCT_NAME VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0),
    STOCK_QUANTITY INT CHECK (STOCK_QUANTITY >= 0),
    CATEGORY_ID INT,
    FOREIGN KEY (CATEGORY_ID) REFERENCES CATEGORY(CATEGORY_ID)
);

CREATE TABLE `ORDER` (
    ORDER_ID INT PRIMARY KEY,
    CUSTOMER_ID INT,
    ORDER_DATE DATE,
    TOTAL_AMOUNT DECIMAL(10,2) CHECK (TOTAL_AMOUNT >= 0),
    ORDER_STATUS VARCHAR(30),
    FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID)
);

CREATE TABLE ORDER_ITEM (
    ORDER_ITEM_ID INT PRIMARY KEY,
    ORDER_ID INT,
    PRODUCT_ID INT,
    Quantity INT CHECK (Quantity > 0),
    Subtotal DECIMAL(10,2) CHECK (Subtotal >= 0),
    FOREIGN KEY (ORDER_ID) REFERENCES `ORDER`(ORDER_ID),
    FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID)
);

CREATE TABLE PAYMENT (
    PAYMENT_ID INT PRIMARY KEY,
    ORDER_ID INT,
    PAYMENT_METHOD VARCHAR(30),
    PAYMENT_DATE DATE,
    PAYMENT_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID) REFERENCES `ORDER`(ORDER_ID)
);

CREATE TABLE DELIVERY (
    DELIVERY_ID INT PRIMARY KEY,
    ORDER_ID INT,
    DELIVERY_ADDRESS VARCHAR(200),
    DELIVERY_DATE DATE,
    DELIVERY_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID) REFERENCES `ORDER`(ORDER_ID)
);

SHOW TABLES;
```

2. Insert Sample Data (DML)

```
-- ========================================
-- File 2: Sample data (DML - INSERT)
-- ========================================
USE ecommerce_order_management;

INSERT INTO CUSTOMER VALUES
(1, 'Ravi',   'ravi@gmail.com',   '9876543210', 'Hyderabad'),
(2, 'Sai',    'sai@gmail.com',    '9876543211', 'Vijayawada'),
(3, 'Kiran',  'kiran@gmail.com',  '9876543212', 'Rajahmundry'),
(4, 'Sneha',  'sneha@gmail.com',  '9876543213', 'Visakhapatnam');

INSERT INTO CATEGORY VALUES
(101, 'Electronics', 'Electronic Products'),
(102, 'Clothing',    'Fashion Products'),
(103, 'Books',       'Educational Books');

INSERT INTO PRODUCT VALUES
(201, 'Headphones',  1500.00, 20, 101),
(202, 'Smart Watch', 2500.00, 15, 101),
(203, 'T-Shirt',      800.00, 30, 102),
(204, 'DBMS Book',    600.00, 25, 103);

INSERT INTO `ORDER` VALUES
(301, 1, '2026-09-28', 1500.00, 'Shipped'),
(302, 2, '2026-09-28', 2500.00, 'Pending'),
(303, 3, '2026-09-29',  800.00, 'Delivered'),
(304, 4, '2026-09-29',  600.00, 'Confirmed');

INSERT INTO ORDER_ITEM VALUES
(401, 301, 201, 1, 1500.00),
(402, 302, 202, 1, 2500.00),
(403, 303, 203, 1,  800.00),
(404, 304, 204, 1,  600.00);

INSERT INTO PAYMENT VALUES
(501, 301, 'UPI',  '2026-09-28', 'Paid'),
(502, 302, 'Card', '2026-09-28', 'Pending'),
(503, 303, 'Cash', '2026-09-29', 'Paid'),
(504, 304, 'UPI',  '2026-09-29', 'Paid');

INSERT INTO DELIVERY VALUES
(601, 301, 'Hyderabad',     '2026-09-30', 'Shipped'),
(602, 302, 'Vijayawada',    '2026-10-01', 'Pending'),
(603, 303, 'Rajahmundry',   '2026-09-30', 'Delivered'),
(604, 304, 'Visakhapatnam', '2026-10-02', 'Confirmed');
```

3. Queries, Update, Delete, View, Alter

```
-- ========================================
-- File 3: Queries (SELECT, JOIN, aggregate,
--         UPDATE, DELETE, VIEW, ALTER)
-- ========================================
USE ecommerce_order_management;

-- ---------- 1. View all tables ----------
SELECT * FROM CUSTOMER;
SELECT * FROM CATEGORY;
SELECT * FROM PRODUCT;
SELECT * FROM `ORDER`;
SELECT * FROM ORDER_ITEM;
SELECT * FROM PAYMENT;
SELECT * FROM DELIVERY;

-- ---------- 2. Basic SELECT queries ----------
-- Products costing more than 1000
SELECT PRODUCT_NAME, Price FROM PRODUCT WHERE Price > 1000;

-- Products sorted by price (high to low)
SELECT * FROM PRODUCT ORDER BY Price DESC;

-- Distinct order statuses
SELECT DISTINCT ORDER_STATUS FROM `ORDER`;

-- Customers whose name starts with 'S'
SELECT * FROM CUSTOMER WHERE CUSTOMER_NAME LIKE 'S%';
```

```sql
-- Customers having an email
SELECT * FROM CUSTOMER WHERE Email IS NOT NULL;

-- ---------- 3. JOIN queries ----------
-- Customer with their orders
SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS
FROM CUSTOMER C
INNER JOIN `ORDER` O ON C.CUSTOMER_ID = O.CUSTOMER_ID;

-- Product with its category
SELECT P.PRODUCT_NAME, P.Price, C.CATEGORY_NAME
FROM PRODUCT P
INNER JOIN CATEGORY C ON P.CATEGORY_ID = C.CATEGORY_ID;

-- Order items with product names
SELECT O.ORDER_ID, P.PRODUCT_NAME, OI.Quantity, OI.Subtotal
FROM `ORDER` O
INNER JOIN ORDER_ITEM OI ON O.ORDER_ID = OI.ORDER_ID
INNER JOIN PRODUCT P ON OI.PRODUCT_ID = P.PRODUCT_ID;

-- Order + payment + delivery status
SELECT O.ORDER_ID, O.ORDER_STATUS, P.PAYMENT_METHOD, P.PAYMENT_STATUS, D.DELIVERY_STATUS
FROM `ORDER` O
INNER JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID
INNER JOIN DELIVERY D ON O.ORDER_ID = D.ORDER_ID;

-- All categories, even those without products (LEFT JOIN)
SELECT C.CATEGORY_NAME, COUNT(P.PRODUCT_ID) AS TOTAL_PRODUCTS
FROM CATEGORY C
LEFT JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
GROUP BY C.CATEGORY_NAME;

-- ---------- 4. Aggregate functions ----------
SELECT COUNT(*)          AS TOTAL_ORDERS         FROM `ORDER`;
SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES          FROM `ORDER`;
SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_AMOUNT FROM `ORDER`;
SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER        FROM `ORDER`;
SELECT MIN(TOTAL_AMOUNT) AS LOWEST_ORDER         FROM `ORDER`;

-- ---------- 5. GROUP BY / HAVING ----------
SELECT ORDER_STATUS, COUNT(*) AS TOTAL_ORDERS
FROM `ORDER`
GROUP BY ORDER_STATUS;

SELECT ORDER_STATUS, COUNT(*) AS TOTAL_ORDERS
FROM `ORDER`
GROUP BY ORDER_STATUS
HAVING COUNT(*) >= 1;

-- ---------- 6. Subquery ----------
-- Products costing more than the average price
SELECT PRODUCT_NAME, Price
FROM PRODUCT
WHERE Price > (SELECT AVG(Price) FROM PRODUCT);

-- ---------- 7. INSERT / UPDATE / DELETE ----------
-- Insert a new category, update it, then delete it
INSERT INTO CATEGORY VALUES (105, 'Home Appliance', 'Home Appliance Products');
UPDATE CATEGORY SET CATEGORY_NAME = 'Home Appliances' WHERE CATEGORY_ID = 105;
SELECT * FROM CATEGORY;
DELETE FROM CATEGORY WHERE CATEGORY_ID = 105;

-- Reduce stock after an order
UPDATE PRODUCT SET STOCK_QUANTITY = STOCK_QUANTITY - 1 WHERE PRODUCT_ID = 201;

-- Update order, payment and delivery status
UPDATE `ORDER`   SET ORDER_STATUS    = 'Delivered' WHERE ORDER_ID = 301;
UPDATE PAYMENT   SET PAYMENT_STATUS  = 'Paid'      WHERE PAYMENT_ID = 502;
UPDATE DELIVERY  SET DELIVERY_STATUS = 'Delivered' WHERE DELIVERY_ID = 601;

-- Delete a product that has no orders (product 204 is used in ORDER_ITEM,
-- so deleting it directly would violate the foreign key)
INSERT INTO PRODUCT VALUES (205, 'Keyboard', 1200.00, 10, 101);
DELETE FROM PRODUCT WHERE PRODUCT_ID = 205;

-- ---------- 8. VIEW ----------
CREATE VIEW Order_Details AS
SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS
```

```
FROM CUSTOMER C
JOIN `ORDER` O ON C.CUSTOMER_ID = O.CUSTOMER_ID;

SELECT * FROM Order_Details;

-- ---------- 9. Constraint demonstration ----------
-- Duplicate email -> fails because of UNIQUE
-- INSERT INTO CUSTOMER VALUES (5, 'Test', 'ravi@gmail.com', '9999999999', 'Hyderabad');
-- Negative price -> fails because of CHECK
-- INSERT INTO PRODUCT VALUES (206, 'Mouse', -10, 10, 101);

-- ---------- 10. ALTER TABLE ----------
ALTER TABLE CUSTOMER ADD Gender VARCHAR(10);
DESC CUSTOMER;
```

Part 2: Screenshots

1. Ddl create customer

```
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE CUSTOMERS (
    CUSTOMERS_ID INT PRIMARY KEY,
    CUSTOMERs_NAME VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Address VARCHAR(200)
);
```

2. Ddl create category

```
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE CATEGORY (
    CATEGORY_ID INT PRIMARY KEY,
    CATEGORY_NAME VARCHAR(100) NOT NULL,
    Description VARCHAR(200)
);
```

3. Ddl create product

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE PRODUCT (
    PRODUCT_ID INT PRIMARY KEY,
    PRODUCT_NAME VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0),
    STOCK_QUANTITY INT CHECK (STOCK_QUANTITY >= 0),
    CATEGORY_ID INT,
    FOREIGN KEY (CATEGORY_ID)
        REFERENCES CATEGORY(CATEGORY_ID)
);
```

4. Ddl create order

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE `ORDER` (
    ORDER_ID INT PRIMARY KEY,
    CUSTOMERS_ID INT,
    ORDER_DATE DATE,
    TOTAL_AMOUNT DECIMAL(10,2) CHECK (TOTAL_AMOUNT >= 0),
    ORDER_STATUS VARCHAR(30),
    FOREIGN KEY (CUSTOMERS_ID)
        REFERENCES CUSTOMERS(CUSTOMERS_ID)
);
```

5. Ddl create order item

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE ORDER_ITEM (
    ORDER_ITEM_ID INT PRIMARY KEY,
    ORDER_ID INT,
    PRODUCT_ID INT,
    Quantity INT CHECK (Quantity > 0),
    Subtotal DECIMAL(10,2) CHECK (Subtotal >= 0),
    FOREIGN KEY (ORDER_ID)
        REFERENCES `ORDER`(ORDER_ID),
    FOREIGN KEY (PRODUCT_ID)
        REFERENCES PRODUCT(PRODUCT_ID)
);
```

6. Ddl create payment

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE PAYMENT (
    PAYMENT_ID INT PRIMARY KEY,
    ORDER_ID INT,
    PAYMENT_METHOD VARCHAR(30),
    PAYMENT_DATE DATE,
    PAYMENT_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID)
        REFERENCES `ORDER`(ORDER_ID)
);
```

7. Ddl create delivery

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE DELIVERY (
    DELIVERY_ID INT PRIMARY KEY,
    ORDER_ID INT,
    DELIVERY_ADDRESS VARCHAR(200),
    DELIVERY_DATE DATE,
    DELIVERY_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID)
        REFERENCES `ORDER`(ORDER_ID)
);
```

8. Insert customer

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO CUSTOMERS
VALUES
(1, 'Ravi', 'ravi@gmail.com', '9876543210', 'Hyderabad'),
(2, 'Anu', 'anu@gmail.com', '9876543211', 'Vijayawada'),
(3, 'Kiran', 'kiran@gmail.com', '9876543212', 'Rajahmundry'),
(4, 'Sneha', 'sneha@gmail.com', '9876543213', 'Visakhapatnam');
```

9. Insert category

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO CATEGORY
VALUES
(101, 'Electronics', 'Electronic Products'),
(102, 'Clothing', 'Fashion Products'),
(103, 'Books', 'Educational Books');
```

10. Insert product

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO PRODUCT
VALUES
(201, 'Headphones', 1500.00, 20, 101),
(202, 'Smart Watch', 2500.00, 15, 101),
(203, 'T-Shirt', 800.00, 30, 102),
(204, 'DBMS Book', 600.00, 25, 103);
```

11. Insert order

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO `ORDER`
VALUES
(301, 1, '2026-09-28', 1500.00, 'Shipped'),
(302, 2, '2026-09-28', 2500.00, 'Pending'),
(303, 3, '2026-09-29', 800.00, 'Delivered'),
(304, 4, '2026-09-29', 600.00, 'Confirmed');
```

12. Insert order item

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO ORDER_ITEM
VALUES
(401, 301, 201, 1, 1500.00),
(402, 302, 202, 1, 2500.00),
(403, 303, 203, 1, 800.00),
(404, 304, 204, 1, 600.00);
```

13. Insert payment

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO PAYMENT
VALUES
(501, 301, 'UPI', '2026-09-28', 'Paid'),
(502, 302, 'Card', '2026-09-28', 'Pending'),
(503, 303, 'Cash', '2026-09-29', 'Paid'),
(504, 304, 'UPI', '2026-09-29', 'Paid');
```

14. Insert delivery

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO DELIVERY
VALUES
(601, 301, 'Hyderabad', '2026-09-30', 'Shipped'),
(602, 302, 'Vijayawada', '2026-10-01', 'Pending'),
(603, 303, 'Rajahmundry', '2026-09-30', 'Delivered'),
(604, 304, 'Visakhapatnam', '2026-10-02', 'Confirmed');
```

15. Select all tables query

```sql
SHOW DATABASES;
USE COLLEGEDB;
SELECT * FROM CUSTOMER;
SELECT * FROM CATEGORY;
SELECT * FROM PRODUCT;
SELECT * FROM `ORDER`;
SELECT * FROM ORDER_ITEM;
SELECT * FROM PAYMENT;
SELECT * FROM DELIVERY;
```

16. Select delivery output

Result Grid | Filter Rows: | Edit: | Export/Import:

DELIVERY_ID	ORDER_ID	DELIVERY_ADDRESS	DELIVERY_DATE	DELIVERY_STATUS
601	301	Hyderabad	2026-09-30	Shipped
602	302	Vijayawada	2026-10-01	Pending
603	303	Rajahmundry	2026-09-30	Delivered
604	304	Visakhapatnam	2026-10-02	Confirmed
NULL	NULL	NULL	NULL	NULL

17. Where price gt 1000

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT PRODUCT_NAME, Price
4  FROM PRODUCT
5  WHERE Price > 1000;
```

Result Grid | Filter Rows:

PRODUCT_NAME	Price
Headphones	1500.00
Smart Watch	2500.00

18. Order by price desc

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT *
4  FROM PRODUCT
5  ORDER BY Price DESC;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

PRODUCT_ID	PRODUCT_NAME	Price	STOCK_QUANTITY	CATEGORY_ID
202	Smart Watch	2500.00	15	101
201	Headphones	1500.00	20	101
203	T-Shirt	800.00	30	102
204	DBMS Book	600.00	25	103
NULL	NULL	NULL	NULL	NULL

19. Distinct order status

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT DISTINCT ORDER_STATUS
4  FROM `ORDER`;
```

Result Grid | Filter Rows:

ORDER_STATUS
Shipped
Pending
Delivered
Confirmed

20. Like customer name

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT *
4  FROM CUSTOMERS
5  WHERE CUSTOMERS_NAME LIKE 'A%';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

CUSTOMERS_ID	CUSTOMERs_NAME	Email	Phone	Address
2	Anu	anu@gmail.com	9876543211	Vijayawada
NULL	NULL	NULL	NULL	NULL

21. Is not null email

```sql
SHOW DATABASES;
USE COLLEGEDB;
SELECT *
FROM CUSTOMERS
WHERE Email IS NOT NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

CUSTOMERS_ID	CUSTOMERs_NAME	Email	Phone	Address
2	Anu	anu@gmail.com	9876543211	Vijayawada
3	Kiran	kiran@gmail.com	9876543212	Rajahmundry
1	Ravi	ravi@gmail.com	9876543210	Hyderabad
4	Sneha	sneha@gmail.com	9876543213	Visakhapatnam
NULL	NULL	NULL	NULL	NULL

22. Join customer orders query

```sql
SHOW DATABASES;
USE COLLEGEDB;
SELECT
    C.CUSTOMER_NAME,
    O.ORDER_ID,
    O.ORDER_DATE,
    O.TOTAL_AMOUNT,
    O.ORDER_STATUS
FROM CUSTOMER C
INNER JOIN `ORDER` O
ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

23. Join product category query

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT
4      P.PRODUCT_NAME,
5      P.Price,
6      C.CATEGORY_NAME
7  FROM PRODUCT P
8  INNER JOIN CATEGORY C
9  ON P.CATEGORY_ID = C.CATEGORY_ID;
```

24. Join product category output

Result Grid | Filter Rows:

PRODUCT_NAME	Price	CATEGORY_NAME
Headphones	1500.00	Electronics
Smart Watch	2500.00	Electronics
T-Shirt	800.00	Clothing
DBMS Book	600.00	Books

25. Join order items query

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT
4      O.ORDER_ID,
5      P.PRODUCT_NAME,
6      OI.Quantity,
7      OI.Subtotal
8  FROM `ORDER` O
9  INNER JOIN ORDER_ITEM OI
10 ON O.ORDER_ID = OI.ORDER_ID
11 INNER JOIN PRODUCT P
12 ON OI.PRODUCT_ID = P.PRODUCT_ID;
```

26. Join order items output

Result Grid | Filter Rows: | Exp

	ORDER_ID	PRODUCT_NAME	Quantity	Subtotal
▶	301	Headphones	1	1500.00
	302	Smart Watch	1	2500.00
	303	T-Shirt	1	800.00
	304	DBMS Book	1	600.00

27. Join order payment delivery query

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT
4      O.ORDER_ID,
5      O.ORDER_STATUS,
6      P.PAYMENT_METHOD,
7      P.PAYMENT_STATUS,
8      D.DELIVERY_STATUS
9  FROM `ORDER` O
10 INNER JOIN PAYMENT P
11 ON O.ORDER_ID = P.ORDER_ID
12 INNER JOIN DELIVERY D
13 ON O.ORDER_ID = D.ORDER_ID;
```

28. Join order payment delivery output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	ORDER_ID	ORDER_STATUS	PAYMENT_METHOD	PAYMENT_STATUS	DELIVERY_STATUS
▶	301	Shipped	UPI	Paid	Shipped
	302	Pending	Card	Pending	Pending
	303	Delivered	Cash	Paid	Delivered
	304	Confirmed	UPI	Paid	Confirmed

29. Aggregate count

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT COUNT(*) AS TOTAL_ORDERS
4  FROM `ORDER`;
```

Result Grid | Filter Rows:

TOTAL_ORDERS
4

30. Aggregate sum

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export:

TOTAL_SALES
5400.00

31. Aggregate avg

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_AMOUNT
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

AVERAGE_ORDER_AMOUNT
1350.000000

32. Aggregate max

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export:

HIGHEST_ORDER
2500.00

33. Aggregate min

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT MIN(TOTAL_AMOUNT) AS LOWEST_ORDER
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export:

LOWEST_ORDER
600.00

34. Group by order status

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT ORDER_STATUS, COUNT(*) AS TOTAL_ORDERS
4  FROM `ORDER`
5  GROUP BY ORDER_STATUS;
```

Result Grid | Filter Rows: | Export: | Wr

ORDER_STATUS	TOTAL_ORDERS
Shipped	1
Pending	1
Delivered	1
Confirmed	1

35. Having order status

```sql
SHOW DATABASES;
USE COLLEGEDB;
SELECT ORDER_STATUS, COUNT(*) AS TOTAL_ORDERS
FROM `ORDER`
GROUP BY ORDER_STATUS
HAVING COUNT(*) >= 1;
```

Result Grid | Filter Rows: | Export: | Wra

ORDER_STATUS	TOTAL_ORDERS
Shipped	1
Pending	1
Delivered	1
Confirmed	1

36. Crud insert category

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO CATEGORY
VALUES (105, 'Home Appliances', 'Home Appliance Products');
```

37. Crud select category

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT *
4  FROM CATEGORY;
```

Result Grid | Filter Rows: | Edit:

CATEGORY_ID	CATEGORY_NAME	Description
101	Electronics	Electronic Products
102	Clothing	Fashion Products
103	Books	Educational Books
105	Home Appliances	Home Appliance Products
NULL	NULL	NULL

38. Crud update category

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  UPDATE CATEGORY
4  SET CATEGORY_NAME = 'Home Appliances'
5  WHERE CATEGORY_ID = 105;
```

39. Crud delete category

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  DELETE FROM CATEGORY
4  WHERE CATEGORY_ID = 105;
```

40. View create

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE VIEW Order_Details AS
SELECT
    C.CUSTOMERS_NAME,
    O.ORDER_ID,
    O.ORDER_DATE,
    O.TOTAL_AMOUNT,
    O.ORDER_STATUS
FROM CUSTOMERS C
JOIN `ORDER` O
ON C.CUSTOMERS_ID = O.CUSTOMERS_ID;
```

41. View select

```sql
SHOW DATABASES;
USE COLLEGEDB;
SELECT *
FROM Order_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CUSTOMERS_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	ORDER_STATUS
Ravi	301	2026-09-28	1500.00	Shipped
Anu	302	2026-09-28	2500.00	Pending
Kiran	303	2026-09-29	800.00	Delivered
Sneha	304	2026-09-29	600.00	Confirmed

42. Constraint insert customer

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO CUSTOMERS
VALUES
(11, 'Test', 'test@gmail.com', '9999999999', 'Hyderabad');
```

43. Constraint foreign key violation

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  INSERT INTO PRODUCT
4    VALUES
5    (205, 'Keyboard', 1200.00, 10, 999);
```

44. Constraint check violation

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  INSERT INTO PRODUCT
4    VALUES
5    (206, 'Mouse', -500.00, 10, 101);
```

45. Alter table desc

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  ALTER TABLE CUSTOMER
4    ADD Gender VARCHAR(10);
5 •  DESC CUSTOMER;
```

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
Name	varchar(30)	YES		NULL	
Referee_ID	int	YES		NULL	
Gender	varchar(10)	YES		NULL	